

You said

Selected files all

The Master Prompt to Copy/Paste into Workik AI Whenever you upload a new controller or file to Workik, paste this exact prompt into the chat sidebar:

Prompt: "Act as a Senior .NET Core Developer and Debugging Expert. I have attached a file/controller from my project. I need a deep-dive, line-by-line, and word-by-word explanation. For every single line, please provide:

Line Purpose: What exactly does this line of code do?

Keyword Breakdown: Break down specific technical keywords used in that line (e.g., async, [FromBody], Task<IActionResult>).

Cross-File References (Debugging Track): Tell me which other projects, folders, or files in a standard Clean Architecture setup this line links to (e.g., Where is the DTO folder? Where is the Interface file? Where is the Service implementation?).

Data Flow: What happens to the data at this exact step?"

FOR EXAMPLE >>>

2. HRMS.API/Controllers/AuthController.cs

Handles user registration, login, and token management.

Line: [ApiController] [Route("api/[controller]")]

Line Purpose: Decorates the class as a Web API controller with a route based on the class name (api/auth).

Keyword Breakdown: [ApiController] (enables automatic model validation and HTTP 400 responses), [Route] (defines the URL path).

Cross-File References: Links to Program.cs routing configuration.

Data Flow: Directs incoming URL requests to this class.

Line: public async Task<IActionResult> Login([FromBody] LoginDto dto)

Line Purpose: Endpoint for user authentication.

Keyword Breakdown: IActionResult (flexible return type for HTTP statuses like Ok, BadRequest), [FromBody] (forces data to be read from the JSON body).

Cross-File References: LoginDto is in HRMS.Application/DTOs/Auth/.

Data Flow: JSON from the client is mapped into the dto object.

Line: var result = await _authService.LoginAsync(dto);

Line Purpose: Calls the business logic layer to verify credentials.

Keyword Breakdown: _authService (the injected interface implementation), await (waits for the service to finish).

Cross-File References: Interface: IAuthService.cs, Implementation: AuthService.cs in HRMS.Application.

Data Flow: Sends user credentials to the service and receives an ApiResponse containing a JWT.

so in my given files you give me according to my given above prompt and like example so we moved one by one files sequencial

>>>IT'S MY APPLICATION FOLDER'S FILE

Gemini said

Here is the deep-dive, line-by-line, and word-by-word breakdown for your application files, following your requested sequence and format.

1. HRMS.Application/Common/ApiResponse.cs

C#

```
namespace HRMS.Application.Common;
```

- **Line Purpose:** Declares the logical namespace to organize the code structure.
- **Keyword Breakdown:** `namespace` (a keyword used to group related classes and prevent naming conflicts).
- **Cross-File References:** Located in the `HRMS.Application` project inside the `Common` folder. Any other file wishing to use this class must include `using HRMS.Application.Common;`.
- **Data Flow:** No runtime data flow; establishes compilation-time scoping.

C#

```
public class ApiResponse<T>
```

- **Line Purpose:** Defines a generic, reusable wrapper class for all API responses to ensure a consistent output format.
- **Keyword Breakdown:** `public` (access modifier making the class visible across all projects), `class` (defines a reference type), `<T>` (a generic type parameter allowing this wrapper to hold any data payload type).
- **Cross-File References:** Used as the standard return type in application service interfaces, implementations, and controllers (e.g., `Task<ApiResponse<TokenResponseDto>>`).
- **Data Flow:** Acts as a generic container that wraps any business logic data payload along with status metadata.

C#

```
public bool Success { get; set; }
```

- **Line Purpose:** A flag property indicating whether the requested operation succeeded or failed.
- **Keyword Breakdown:** `bool` (boolean data type: `true` or `false`), `{ get; set; }` (auto-implemented property syntax creating a hidden backing field).
- **Cross-File References:** Inspected by client applications (Frontend) and API controllers to check the operational outcome.
- **Data Flow:** Populated by factory methods (`SuccessResult` / `Failure`) and serialized into JSON for the HTTP response body.

C#

```
public string Message { get; set; } = string.Empty;
```

- **Line Purpose:** A feedback message string describing the outcome of the operation.
- **Keyword Breakdown:** `string` (sequence of characters), `= string.Empty;` (property initializer to prevent null-reference issues by defaulting to `""`).
- **Cross-File References:** Uses error or success strings, frequently pulling values from `AppConstants.Messages`.
- **Data Flow:** Carries status messages back to the user interface.

C#

```
public T? Data { get; set; }
```

- **Line Purpose:** The actual payload returned by the operation, marked as nullable because failures don't return data.
- **Keyword Breakdown:** `T?` (nullable reference or value type based on the generic parameter `T`).
- **Cross-File References:** Dynamically matches any DTO type passed from the services (e.g., `EmployeeResponseDto`, `TokenResponseDto`).
- **Data Flow:** Transports the successfully processed data object out of the Application layer to the API Controllers.

C#

```
public List<string> Errors { get; set; } = new();
```

- **Line Purpose:** Holds a collection of specific error messages if the operation fails.
- **Keyword Breakdown:** `List<string>` (a strongly-typed, dynamic array list of strings), `= new();` (C# 9 target-typed new expression to initialize an empty list).
- **Cross-File References:** Populated during validation failures or exceptional flows in application services.
- **Data Flow:** Aggregates error details to send back to the frontend.

C#

```
public static ApiResponse<T> SuccessResult(T data, string message = "Operation successful.")
    => new() { Success = true, Message = message, Data = data };
```

- **Line Purpose:** A factory method used to cleanly instantiate a successful API response object.
- **Keyword Breakdown:** `static` (method belongs to the class itself, not an instance), `=>` (expression-bodied member syntax used for brief method definitions).
- **Cross-File References:** Called directly inside application service implementations when an operation completes successfully.
- **Data Flow:** Receives the data payload and an optional message, constructs an `ApiResponse<T>`, sets `Success` to `true`, and passes it back.

C#

```
public static ApiResponse<T> Failure(string message, List<string>? errors = null)
    => new() { Success = false, Message = message, Errors = errors ?? new List<string>() };
```

- **Line Purpose:** A factory method used to instantiate a failed API response object.
- **Keyword Breakdown:** `?` (nullable reference type marker allowing `errors` to be null), `??` (null-coalescing operator; substitutes a new list if `errors` is null).
- **Cross-File References:** Invoked across application services when validation fails or data isn't found.
- **Data Flow:** Sets `Success` to `false`, maps the error messages, and ignores the `Data` field.

2. HRMS.Application/Common/AppConstants.cs

C#

```
namespace HRMS.Application.Common;
public static class AppConstants
{
```

- **Line Purpose:** Defines a static, global container class for application-wide string constants.
- **Keyword Breakdown:** `static class` (cannot be instantiated or inherited; can only contain static members).
- **Cross-File References:** References compiled globally across the `Application` and `API` projects.
- **Data Flow:** Static declaration compile-time lookup; no active run-time data mutations.

C#

```
public static class Roles
{
                                                                    public const string Admin = "Admin";
                                                                    public const string HR = "HR";
}
```

- **Line Purpose:** Groups role names into a central static sub-class to avoid magic strings.
- **Keyword Breakdown:** `const` (compile-time constant value that cannot be altered).
- **Cross-File References:** Used by `TokenResponseDto.cs`, authorization handlers, and controller attributes like `[Authorize(Roles = AppConstants.Roles.Admin)]`.
- **Data Flow:** Values are injected into JWT tokens during authentication and checked during permission checks.

C#

```
public static class Messages
{
                                                                    public const string UserNotFound = "User not found.";
                                                                    public const string InvalidCredentials = "Invalid email or password.";
                                                                    public const string OrganizationNotFound = "Organization not found.";
                                                                    public const string EmployeeNotFound = "Employee not found.";
                                                                    public const string Unauthorized = "You are not authorized to perform this action.";
                                                                    public const string UserAlreadyExists = "A user with this email already exists.";
                                                                    public const string OrganizationAlreadyExists = "You already belong to an organization. Ca
                                                                    public const string InvalidToken = "Invalid or expired token.";
                                                                    public const string NoOrganization = "User does not belong to any organization.";
}
```

- **Line Purpose:** Centralized storage for system error and feedback messages.
- **Keyword Breakdown:** `const string` (immutable character arrays embedded directly into the assembly at compile time).
- **Cross-File References:** Heavily used by validation classes and application service implementations when returning `ApiResponse<T>.Failure()`.
- **Data Flow:** Selected message strings are fed into `ApiResponse<T>.Message` parameters upon logic failures.

3. HRMS.Application/DTOs/Auth/LoginDto.cs

C#

```
using System.ComponentModel.DataAnnotations;
namespace HRMS.Application.DTOs.Auth;
```

- **Line Purpose:** Imports the data annotations library and creates the specific namespace for authentication DTOs.
- **Keyword Breakdown:** `using` (directive importing namespace functionalities into scope).
- **Cross-File References:** Part of `HRMS.Application` in the `DTOs/Auth` folder.
- **Data Flow:** Evaluated when processing requests coming directly into controllers.

C#

```
public class LoginDto
{
```

- **Line Purpose:** Data Transfer Object holding user input required to perform a login operation.
- **Cross-File References:** Instantiated automatically by ASP.NET Core framework when requests target an authentication controller endpoint.

C#

```
[Required]
[EmailAddress]
public string Email { get; set; } = string.Empty;
```

- **Line Purpose:** Field property accepting the user's email, guarded by validation attributes.
- **Keyword Breakdown:** `[Required]` (validation attribute enforcing a non-null/non-empty string value), `[EmailAddress]` (validates format complies with email standard structures).
- **Cross-File References:** Directly utilized by `AuthController.cs` validation middleware.
- **Data Flow:** Captures user input JSON payload `"email": "..."` from request body, checks constraints, and maps to the property.

C#

```
[Required]
public string Password { get; set; } = string.Empty;
```

- **Line Purpose:** Field property accepting the user's cleartext password.
- **Cross-File References:** Evaluated in authentication services against hashed database fields.
- **Data Flow:** Maps incoming client request string into this variable to pass over to security layers.

4. HRMS.Application/DTOs/Auth/RefreshTokenRequestDto.cs

C#

```
namespace HRMS.Application.DTOs.Auth;
public class RefreshTokenRequestDto
{
    public string Token { get; set; } = string.Empty;
}
```

- **Line Purpose:** Data Transfer Object structure designed to hold a refresh token value for renewing sessions or logging out.
- **Cross-File References:** Maps requests inside `AuthController.cs` and moves data to authentication service operations.
- **Data Flow:** Bound from the incoming request body JSON property `token`, allowing the system to inspect database records for validation/revocation.

5. HRMS.Application/DTOs/Auth/RegisterDto.cs

C#

```
using System.ComponentModel.DataAnnotations;
namespace HRMS.Application.DTOs.Auth;
public class RegisterDto
{
    [Required]
    public string FullName { get; set; } = string.Empty;

    [Required]
    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Required]
    [MinLength(6)]
    public string Password { get; set; } = string.Empty;
}
```

- **Line Purpose:** Data Transfer Object defining user parameters needed for creating new application accounts.
- **Keyword Breakdown:** `[MinLength(6)]` (validation rule rejecting incoming passwords shorter than six characters).
- **Cross-File References:** Targets registrations inside controllers, which forwards properties to create user entity records in infrastructure databases.
- **Data Flow:** Incoming registration parameters are automatically validated against these parameters. On success, fields map forward to generate standard application accounts.

6. HRMS.Application/DTOs/Auth/TokenResponseDto.cs

C#

```
namespace HRMS.Application.DTOs.Auth;
public class TokenResponseDto
```

- **Line Purpose:** Complete collection data structure containing context details and authorization strings generated after successful authentication events.
- **Cross-File References:** Returned inside an `ApiResponse<TokenResponseDto>` from services back up to frontend clients.

C#

```
public string AccessToken { get; set; } = string.Empty;
public string RefreshToken { get; set; } = string.Empty;
public DateTime ExpiresAt { get; set; }
```

- **Line Purpose:** Core parameters managing the authentication lifetime (JWT string, tracking refresh code, explicit expiration timestamp).
- **Keyword Breakdown:** `DateTime` (system struct representing dates and times).
- **Data Flow:** Constructed by security token generation components and passed to client storage headers.

C#

```
public string FullName { get; set; } = string.Empty;
public string Email { get; set; } = string.Empty;
public Guid UserId { get; set; }
public Guid? OrganizationId { get; set; }
public List<string> Roles { get; set; } = new();
```

- **Line Purpose:** Context metadata attached along with token payloads to allow applications to customize sessions instantly without immediately refetching details.
- **Keyword Breakdown:** `Guid` (Globally Unique Identifier struct used for DB keys), `Guid?` (nullable unique identifier; permits users not currently bound to organizations).
- **Data Flow:** Extracted from persisted infrastructure entity models during login validation and mapped directly into this return DTO object.

7. HRMS.Application/DTOs/Employee/CreateEmployeeDto.cs

C#

```
namespace HRMS.Application.DTOs.Employee;
public class CreateEmployeeDto
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string PhoneNumber { get; set; } = string.Empty;
    public string Address { get; set; } = string.Empty;
}
```

- **Line Purpose:** Defines the explicit fields required by HR personnel users to register a new employee record.
- **Cross-File References:** Used by `EmployeeController.cs` and passed into the `IEmployeeService` pipeline.
- **Data Flow:** Gathers text entries from incoming JSON body models, packaging them together for processing into domain entities.

8. HRMS.Application/DTOs/Employee/EmployeeResponseDto.cs

C#

```
namespace HRMS.Application.DTOs.Employee;
public class EmployeeResponseDto
{
                                                                    public Guid Id { get; set; }
                                                                    public string FirstName { get; set; } = string.Empty;
                                                                    public string LastName { get; set; } = string.Empty;
                                                                    public string Email { get; set; } = string.Empty;
                                                                    public string PhoneNumber { get; set; } = string.Empty;
                                                                    public string Address { get; set; } = string.Empty;
                                                                    public Guid CreatedBy { get; set; }
                                                                    public string CreatedByName { get; set; } = string.Empty;
                                                                    public Guid OrganizationId { get; set; }
}
```

- **Line Purpose:** Standardized output format for returning complete details of an employee record, safely preventing exposure of system entities directly.
- **Cross-File References:** Returned by employee querying functions within interfaces and controller routes.
- **Data Flow:** Maps data pulled out from underlying persistence stores into this structure before dispatching outward to external callers.

9. HRMS.Application/DTOs/Employee/UpdateEmployeeDto.cs

C#

```
namespace HRMS.Application.DTOs.Employee;
public class UpdateEmployeeDto
{
                                                                    public string FirstName { get; set; } = string.Empty;
                                                                    public string LastName { get; set; } = string.Empty;
                                                                    public string Email { get; set; } = string.Empty;
                                                                    public string PhoneNumber { get; set; } = string.Empty;
                                                                    public string Address { get; set; } = string.Empty;
}
```

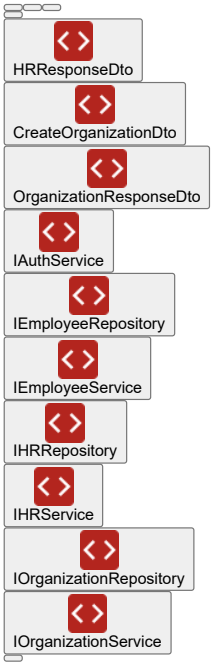
- **Line Purpose:** Carries data changes explicitly intended to patch or replace fields on existing employee entities.
- **Cross-File References:** Placed within parameters of update endpoints inside the application layer.
- **Data Flow:** Supplies values to modify matched records identified by target identifiers.

10. HRMS.Application/DTOs/HR/CreateHRDto.cs

C#

```
namespace HRMS.Application.DTOs.HR;
public class CreateHRDto
{
                                                                    public string FullName { get; set; } = string.Empty;
                                                                    public string Email { get; set; } = string.Empty;
                                                                    public string Password { get; set; } = string.Empty;
}
```

- **Line Purpose:** Form data blueprint utilized exclusively by administrative roles to inject new HR profile credentials under a designated organization scope.
- **Cross-File References:** Handled by administration service implementations.
- **Data Flow:** Transports user details across application logic scopes to establish specialized security and group context accounts.



Gemini said

Here is the sequential, line-by-line breakdown for this next batch of files from your application layer, keeping the exact same structure as before.

1. HRMS.Application/DTOs/HR/HRResponseDto.cs

C#

```
namespace HRMS.Application.DTOs.HR;
```

- **Line Purpose:** Declares the specific namespace for HR-related Data Transfer Objects.
- **Keyword Breakdown:** `namespace` (organizes code structure logically and prevents naming collisions).
- **Cross-File References:** Located in the `HRMS.Application` project under `DTOs/HR`. It will be referenced by `IHRService.cs` and your controllers.
- **Data Flow:** No active data runtime flow; establishes compilation scope.

C#

```
public class HRResponseDto
```

- **Line Purpose:** Defines the structured data blueprint returned to the client containing safe profile information about an HR user.
- **Keyword Breakdown:** `public` (accessible anywhere outside the project), `class` (reference type definition).
- **Cross-File References:** Used as the wrapped return data type in `IHRService.cs` (`ApiResponse<HRResponseDto>`).
- **Data Flow:** Holds the safe out-facing user profile information mapped from internal application user data structures.

C#

```
public Guid Id { get; set; }
public string FullName { get; set; } = string.Empty;
public string Email { get; set; } = string.Empty;
public Guid? OrganizationId { get; set; }
public DateTime CreatedAt { get; set; }
```

- **Line Purpose:** Declares specific read-write fields exposed to the frontend about an HR employee profile.
- **Keyword Breakdown:** `Guid?` (nullable unique identifier; permits an HR profile to temporarily exist outside a designated organization), `DateTime` (system struct representing exact chronological timestamps).
- **Cross-File References:** Maps values natively pulled from `ApplicationUser` domain records.
- **Data Flow:** Receives database values from the service layer mapping logic and transfers them out to your API JSON serializers.

2. HRMS.Application/DTOs/Organization/CreateOrganizationDto.cs

C#

```
namespace HRMS.Application.DTOs.Organization;
public class CreateOrganizationDto
{
                                                                        public string Name { get; set; } = string.Empty;
                                                                        public string Address { get; set; } = string.Empty;

}
```

- **Line Purpose:** Collects essential input text fields needed from an authenticated user to instantiate a new organization registry profile.
- **Cross-File References:** Handled as an incoming argument model inside `IOrganizationService.cs`.
- **Data Flow:** Accepts client-side JSON inputs via controller bodies and forwards the request data forward to be wrapped into a structural `Organization` entity.

3. HRMS.Application/DTOs/Organization/OrganizationResponseDto.cs

C#

```
namespace HRMS.Application.DTOs.Organization;
public class OrganizationResponseDto
{
                                                                        public Guid Id { get; set; }
                                                                        public string Name { get; set; } = string.Empty;
                                                                        public string Address { get; set; } = string.Empty;
                                                                        public DateTime CreatedAt { get; set; }
}
```

- **Line Purpose:** Packages safe, non-sensitive organizational details to output to frontend users following successful lookups or additions.
- **Cross-File References:** Formats collection response methods across `IOrganizationService.cs`.
- **Data Flow:** Converts structural entities fetched by underlying data drivers into serialized strings suitable for consumer-side clients.

4. HRMS.Application/Interfaces/IAuthService.cs

C#

```
using HRMS.Application.Common;
using HRMS.Application.DTOs.Auth;
```

- **Line Purpose:** Imports the shared responses container and authentication transfer models into scope.
- **Cross-File References:** References `ApiResponse<T>` from `Common/` and standard registration/login DTOs.

C#

```
namespace HRMS.Application.Interfaces;
public interface IAuthService
{
```

- **Line Purpose:** Establishes the business logic contract specifying all accessible authentication procedures.
- **Keyword Breakdown:** `interface` (a contract declaration containing method signatures without concrete operational implementations).
- **Cross-File References:** Implemented within your Infrastructure or Core Application layer (e.g., `AuthService.cs`) and injected directly into `AuthController.cs`.

C#

```
    Task<ApiResponse<bool>> RegisterAsync(RegisterDto dto);
```

- **Line Purpose:** Signature for registering a user profile asynchronously.
- **Keyword Breakdown:** `Task<...>` (represents an asynchronous operation that returns a value), `Async` (naming convention signifying non-blocking code).
- **Data Flow:** Takes a `RegisterDto` down into the implementation pipeline and passes a success/failure boolean flag back up to the controller.

C#

```
    Task<ApiResponse<TokenResponseDto>> LoginAsync(LoginDto dto);
    Task<ApiResponse<TokenResponseDto>> RefreshTokenAsync(string token);
    Task<ApiResponse<bool>> LogoutAsync(string token);
}
```

- **Line Purpose:** Outlines method contracts for executing authentication validation, refreshing active sessions, or invalidating user tokens.
- **Data Flow:** Moves active token strings or login credentials down into security logic, returning authenticated JWT response details or operation status indicators up to the user.

5. HRMS.Application/Interfaces/IEmployeeRepository.cs

C#

```
using HRMS.Domain.Entities;
namespace HRMS.Application.Interfaces;
```

- **Line Purpose:** Declares the persistence contract managing CRUD processes for Employee entities, linking Application to Domain layers.
- **Cross-File References:** References the domain model definitions (`Employee`) located in `HRMS.Domain/Entities/`. Implemented in your Infrastructure layer (e.g., `EmployeeRepository.cs` via Entity Framework Core).

C#

```
public interface IEmployeeRepository
{
                                                                        Task<Employee> CreateAsync(Employee employee);
    Task<Employee?> GetByIdAsync(Guid id);
    Task<List<Employee>> GetAllAsync(Guid? organizationId, Guid? createdBy);
                                                                        Task<Employee> UpdateAsync(Employee employee);
                                                                        Task DeleteAsync(Employee employee);
}
```

- **Line Purpose:** Outlines low-level asynchronous database interaction tasks dedicated exclusively to managing employee records.
- **Keyword Breakdown:** `Employee?` (indicates that a query searching for a specific identifier might safely return null if no matching record is found).
- **Data Flow:** Acts as the barrier crossing directly into your physical infrastructure storage; accepts and returns domain entity objects directly.

6. HRMS.Application/Interfaces/IHRRepository.cs

C#

```
using HRMS.Domain.Entities;
namespace HRMS.Application.Interfaces;

public interface IHRRepository
{
    Task<List<ApplicationUser>> GetHRsByOrganizationAsync(Guid organizationId);
    Task<ApplicationUser?> GetHRByIdAsync(Guid hrId);
}
```

- **Line Purpose:** Sets up storage retrieval requirements for fetching specialized application user accounts that hold administrative or HR roles.
- **Cross-File References:** References `ApplicationUser` entity from the `HRMS.Domain` layer.
- **Data Flow:** Pulls filtered raw database user accounts scoped to an organization's identifier so that services can convert them into DTO representations.

7. HRMS.Application/Interfaces/IHRService.cs

C#

```
using HRMS.Application.Common;
using HRMS.Application.DTOs.HR;
namespace HRMS.Application.Interfaces;

public interface IHRService
{
    Task<ApiResponse<HRResponseDto>> CreateHRAsync(CreateHRDto dto, Guid adminUserId);
    Task<ApiResponse<List<HRResponseDto>>> GetAllHRsAsync(Guid adminUserId);
    Task<ApiResponse<bool>> RemoveHRAsync(Guid hrId, Guid adminUserId);
}
```

- **Line Purpose:** Outlines high-level administrative orchestration rules governing the provisioning and deletion of HR accounts.
- **Cross-File References:** Consumes `CreateHRDto`, `HRResponseDto`, and handles the business rule mappings behind your API presentation routes.
- **Data Flow:** Accepts DTO payloads along with contextual security details (`adminUserId`), uses repositories to apply changes to the database, and returns standard `ApiResponse` packages.

8. HRMS.Application/Interfaces/IOrganizationRepository.cs

C#

```
using HRMS.Domain.Entities;
namespace HRMS.Application.Interfaces;

public interface IOrganizationRepository
{
                                                                        Task<Organization> CreateAsync(Organization organization);
    Task<List<Organization>> GetAllAsync();
    Task<Organization?> GetByIdAsync(Guid id);
}
```

- **Line Purpose:** Defines database storage capability requirements for writing and querying primary organization records.

- **Cross-File References:** Interfaces directly between infrastructure persistence code blocks and your core `Organization` domain entities.
- **Data Flow:** Persists newly instantiated tenant records and surfaces data arrays from physical database tables.

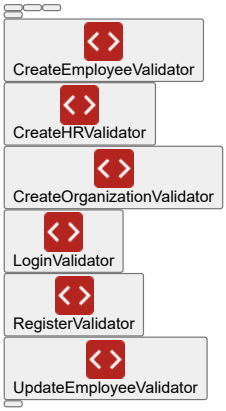
9. HRMS.Application/Interfaces/IOrganizationService.cs

C#

```
using HRMS.Application.Common;
using HRMS.Application.DTOs.Organization;
namespace HRMS.Application.Interfaces;

public interface IOrganizationService
{
    Task<ApiResponse<OrganizationResponseDto>> CreateOrganizationAsync(CreateOrganizationDto dto, Guid userId);
    Task<ApiResponse<List<OrganizationResponseDto>>> GetAllOrganizationsAsync();
}
```

- **Line Purpose:** Exposes the application-level workspace routines handling tenant/company creation and list aggregation.
- **Cross-File References:** Utilized inside company registration endpoints; consumes organization DTOs.
- **Data Flow:** Coordinates domain workflows by accepting validation models, verifying constraints via repositories, and assembling presentation-safe DTO outputs.



Gemini said

Here is the sequential, deep-dive, line-by-line breakdown for the validation classes in your application layer, matching your exact requested template and structure.

1. HRMS.Application/Validators/CreateEmployeeValidator.cs

C#

```
using FluentValidation;
using HRMS.Application.DTOs.Employee;
```

- **Line Purpose:** Imports the required programmatic dependencies for constructing data validators and references employee DTO definitions.
- **Keyword Breakdown:** `using` (directive used to include types from external namespaces into the file's scope).
- **Cross-File References:** Pulls `AbstractValidator` from the FluentValidation library and `CreateEmployeeDto` from the `HRMS.Application/DTOs/Employee` directory.

C#

```
namespace HRMS.Application.Validators;
public class CreateEmployeeValidator : AbstractValidator<CreateEmployeeDto>
{
```

- **Line Purpose:** Defines a data validation engine tailored specifically to intercept and check properties on employee creation requests.
- **Keyword Breakdown:** `class` (reference type declaration), `:` (inheritance syntax operator), `AbstractValidator<T>` (the generic base class from FluentValidation that provides rule-building methods).
- **Cross-File References:** Automatically wired into the request pipeline or invoked manually inside your `EmployeeController` or application services before saving data.
- **Data Flow:** Receives a hydrated `CreateEmployeeDto` reference into memory and runs individual validation operations against it.

C#

```
    public CreateEmployeeValidator()
    {
```

- **Line Purpose:** Default constructor defining the specific data validation rules that run immediately upon instantiation.
- **Keyword Breakdown:** `public` (makes the constructor accessible to dependency injection containers), `CreateEmployeeValidator()` (the constructor initialization method).

C#

```
        RuleFor(x => x.FirstName)
            .NotEmpty().WithMessage("First name is required.")
            .MaximumLength(100).WithMessage("First name cannot exceed 100 characters.")
            .Matches(@"^[A-Za-z\s]+$").WithMessage("First name must contain only letters and spaces.");
```

- **Line Purpose:** Configures validation constraints for the incoming `FirstName` string property.
- **Keyword Breakdown:** `RuleFor()` (FluentValidation entry method to target properties), `NotEmpty()` (checks for null, empty, or whitespace strings), `MaximumLength()` (bounds text length), `Matches()` (enforces validation against a Regular Expression).
- **Cross-File References:** Links to the `FirstName` property inside `CreateEmployeeDto.cs`.
- **Data Flow:** Evaluates the `FirstName` property string. If validation fails, it blocks further execution and adds the specified custom messages to the validation error collection.

C#

```
        RuleFor(x => x.LastName)
            .NotEmpty().WithMessage("Last name is required.")
            .MaximumLength(100).WithMessage("Last name cannot exceed 100 characters.")
            .Matches(@"^[A-Za-z\s]+$").WithMessage("Last name must contain only letters and spaces.");
```

- **Line Purpose:** Validates data format rules for the `LastName` string property.
- **Cross-File References:** Enforces rules on `LastName` inside `CreateEmployeeDto.cs`.
- **Data Flow:** Evaluates character formatting. Regular expression matches check that only valid text letters are passed down to infrastructure services.

C#

```
        RuleFor(x => x.Email)
            .NotEmpty().WithMessage("Email is required.")
            .EmailAddress().WithMessage("Please enter a valid email address.");
```

- **Line Purpose:** Configures formatting safety policies explicitly for the `Email` payload property.
- **Keyword Breakdown:** `EmailAddress()` (built-in validation rule confirming the presence of a standardized `@` domain structure).
- **Cross-File References:** Corresponds to the `Email` text property inside `CreateEmployeeDto.cs`.

C#

```
        RuleFor(x => x.PhoneNumber)
            .NotEmpty().WithMessage("Phone number is required.")
            .Matches(@"^\d+$").WithMessage("Phone number must contain digits only.")
            .MaximumLength(20).WithMessage("Phone number cannot exceed 20 digits.");
```

- **Line Purpose:** Validates that input fields provided for the phone number contain only numeric data values.
- **Cross-File References:** Links directly to the `PhoneNumber` string property inside `CreateEmployeeDto.cs`.

C#

```
        RuleFor(x => x.Address)
            .NotEmpty().WithMessage("Address is required.")
            .MaximumLength(500).WithMessage("Address cannot exceed 500 characters.");
    }
}
```

- **Line Purpose:** Sets upper bound length limits on text data for the physical location parameter string.
- **Cross-File References:** Ties directly into `Address` within `CreateEmployeeDto.cs`.

2. HRMS.Application/Validators/CreateHRValidator.cs

C#

```
using FluentValidation;
using HRMS.Application.DTOs.HR;
namespace HRMS.Application.Validators;

public class CreateHrValidator : AbstractValidator<CreateHRDto>
{
```

- **Line Purpose:** Initializes validation tracking structures used to protect user management methods when generating new human resources credentials.
- **Cross-File References:** Maps structural constraints over fields specified inside `CreateHRDto.cs`.

C#

```
    public CreateHrValidator()
    {
        RuleFor(x => x.FullName)
            .NotEmpty().WithMessage("Full name is required.")
            .MaximumLength(200).WithMessage("Full name cannot exceed 200 characters.")
            .Matches(@"^[A-Za-z\s]+$").WithMessage("Full name must contain only letters and spaces.");
```

- **Line Purpose:** Validates the descriptive `FullName` entry parameter for new system users.
- **Cross-File References:** References `FullName` inside `CreateHRDto.cs`.

C#

```
        RuleFor(x => x.Email)
            .NotEmpty().WithMessage("Email is required.")
            .EmailAddress().WithMessage("Please enter a valid email address.");
```

- **Line Purpose:** Validates standard internet routing compliance for the designated login username target.
- **Cross-File References:** Targets the `Email` parameter from `CreateHRDto.cs`.

C#

```
        RuleFor(x => x.Password)
            .NotEmpty().WithMessage("Password is required.")
            .MinimumLength(8).WithMessage("Password must be at least 8 characters.")
            .Matches("[A-Z]").WithMessage("Password must contain at least one uppercase letter.")
            .Matches("[a-z]").WithMessage("Password must contain at least one lowercase letter.")
            .Matches("[0-9]").WithMessage("Password must contain at least one digit.")
            .Matches(@"[\W_]").WithMessage("Password must contain at least one special character.");
    }
}
```

- **Line Purpose:** Implements strong password security checks to ensure passwords meet complexity requirements before account creation.
- **Keyword Breakdown:** `MinimumLength(8)` (rejects password inputs shorter than eight characters), `Matches("[A-Z]")` (requires an uppercase character), `[\W_]` (regular expression lookahead matching non-alphanumeric special symbols).
- **Cross-File References:** Validates the input payload bound to `Password` inside `CreateHRDto.cs`.

- **Data Flow:** Scans the password sequence string in memory. If any complexity checks fail, execution halts and returns structured errors before hitting user creation logic.

3. HRMS.Application/Validators/CreateOrganizationValidator.cs

C#

```
using FluentValidation;
using HRMS.Application.DTOs.Organization;
namespace HRMS.Application.Validators;

public class CreateOrganizationValidator : AbstractValidator<CreateOrganizationDto>
{
```

- **Line Purpose:** Class definition enforcing configuration guidelines for newly registered business entity tenants.
- **Cross-File References:** Validates property fields originating from `CreateOrganizationDto.cs`.

C#

```
    public CreateOrganizationValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("Organization name is required.")
            .MaximumLength(200).WithMessage("Organization name cannot exceed 200 characters.")
            .Must(name => name.Any(char.IsLetter))
            .WithMessage("Organization name must contain at least one alphabet.");
```

- **Line Purpose:** Validates tenant entity names, ensuring they contain text characters rather than just numbers or symbols.
- **Keyword Breakdown:** `Must()` (defines a custom predicate function for bespoke validation logic), `name.Any(char.IsLetter)` (LINQ query checking that at least one character in the string is an alphabet letter).
- **Cross-File References:** Directly tracks `Name` property inputs handled by `CreateOrganizationDto.cs`.

C#

```
        RuleFor(x => x.Address)
            .NotEmpty().WithMessage("Organization address is required.")
            .MaximumLength(500).WithMessage("Organization address cannot exceed 500 characters.");
    }
}
```

- **Line Purpose:** Sets data rules for the corporate office string argument.
- **Cross-File References:** Checks `Address` constraints on `CreateOrganizationDto.cs`.

4. HRMS.Application/Validators/LoginValidator.cs

C#

```
using FluentValidation;
using HRMS.Application.DTOs.Auth;
namespace HRMS.Application.Validators;

public class LoginValidator : AbstractValidator<LoginDto>
{
```

- **Line Purpose:** Provides basic structural validation for the application's login entry routes.
- **Cross-File References:** Targets parameters managed by `LoginDto.cs`.

C#

```
    public LoginValidator()
    {
        RuleFor(x => x.Email)
            .NotEmpty().WithMessage("Email is required.")
            .EmailAddress().WithMessage("Please enter a valid email address.");

        RuleFor(x => x.Password)
            .NotEmpty().WithMessage("Password is required.");
    }
}
```

- **Line Purpose:** Ensures authentication requests contain non-empty credentials and valid email strings before processing.
- **Cross-File References:** Directly evaluates field values coming from `LoginDto.cs`.
- **Data Flow:** Prevents authentication engines from querying databases with missing or malformed email patterns.

5. HRMS.Application/Validators/RegisterValidator.cs

C#

```
using FluentValidation;
using HRMS.Application.DTOs.Auth;
namespace HRMS.Application.Validators;

public class RegisterValidator : AbstractValidator<RegisterDto>
{
```

- **Line Purpose:** Validates new user parameters during application self-registration flows.
- **Cross-File References:** Applies security rules onto fields coming from `RegisterDto.cs`.

C#

```
    public RegisterValidator()
    {
        RuleFor(x => x.FullName)
            .NotEmpty().WithMessage("Full name is required.")
            .MaximumLength(200).WithMessage("Full name cannot exceed 200 characters.")
            .Matches(@"^[A-Za-z\s]+$").WithMessage("Full name must contain only letters and spaces.");
```

```
        RuleFor(x => x.Email)
            .NotEmpty().WithMessage("Email is required.")
            .EmailAddress().WithMessage("Please enter a valid email address.");
```

- **Line Purpose:** Enforces format policies for a registering user's name and email parameters.
- **Cross-File References:** Maps validations directly onto fields defined in `RegisterDto.cs`.

C#

```
        RuleFor(x => x.Password)
            .NotEmpty().WithMessage("Password is required.")
            .MinimumLength(8).WithMessage("Password must be at least 8 characters.")
            .Matches("[A-Z]").WithMessage("Password must contain at least one uppercase letter.")
            .Matches("[a-z]").WithMessage("Password must contain at least one lowercase letter.")
            .Matches("[0-9]").WithMessage("Password must contain at least one digit.")
            .Matches(@"[\W_]").WithMessage("Password must contain at least one special character.");
    }
}
```

- **Line Purpose:** Enforces public account security metrics on user-submitted password inputs.
- **Cross-File References:** Validates the incoming string targeting `Password` inside `RegisterDto.cs`.

6. HRMS.Application/Validators/UpdateEmployeeValidator.cs

C#

```
using FluentValidation;
using HRMS.Application.DTOs.Employee;
namespace HRMS.Application.Validators;

public class UpdateEmployeeValidator : AbstractValidator<UpdateEmployeeDto>
{
```

- **Line Purpose:** Validates data models sent when modifying an existing employee's system record.
- **Cross-File References:** Monitors incoming fields provided by `UpdateEmployeeDto.cs`.

C#

```
    public UpdateEmployeeValidator()
    {
        RuleFor(x => x.FirstName)
            .NotEmpty().WithMessage("First name is required.")
            .MaximumLength(100).WithMessage("First name cannot exceed 100 characters.")
            .Matches(@"^[A-Za-z\s]+$").WithMessage("First name must contain only letters and spaces.");

        RuleFor(x => x.LastName)
            .NotEmpty().WithMessage("Last name is required.")
            .MaximumLength(100).WithMessage("Last name cannot exceed 100 characters.")
            .Matches(@"^[A-Za-z\s]+$").WithMessage("Last name must contain only letters and spaces.");
```

- **Line Purpose:** Validates that name update strings match alphabetical constraints.
- **Cross-File References:** Targets the `FirstName` and `LastName` fields inside `UpdateEmployeeDto.cs`.

C#

```
        RuleFor(x => x.Email)
            .NotEmpty().WithMessage("Email is required.")
            .EmailAddress().WithMessage("Please enter a valid email address.");

        RuleFor(x => x.PhoneNumber)
            .NotEmpty().WithMessage("Phone number is required.")
            .Matches(@"^\d+$").WithMessage("Phone number must contain digits only.")
            .MaximumLength(20).WithMessage("Phone number cannot exceed 20 digits.");

        RuleFor(x => x.Address)
            .NotEmpty().WithMessage("Address is required.")
            .MaximumLength(500).WithMessage("Address cannot exceed 500 characters.");
    }
}
```

- **Line Purpose:** Ensures that updated communication strings and location fields are formatted correctly before being written to persistence stores.
- **Cross-File References:** Evaluates values found in `UpdateEmployeeDto.cs`.
- **Data Flow:** Ensures updated contact strings pass corporate data format rules before being forwarded to data repositories.